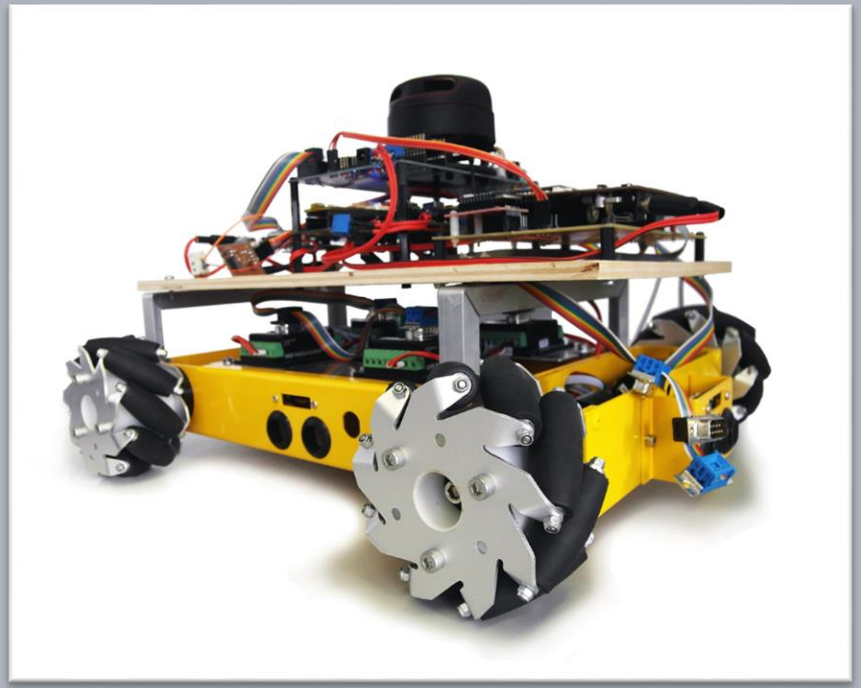
Advanced Embedded Architectures and Applications

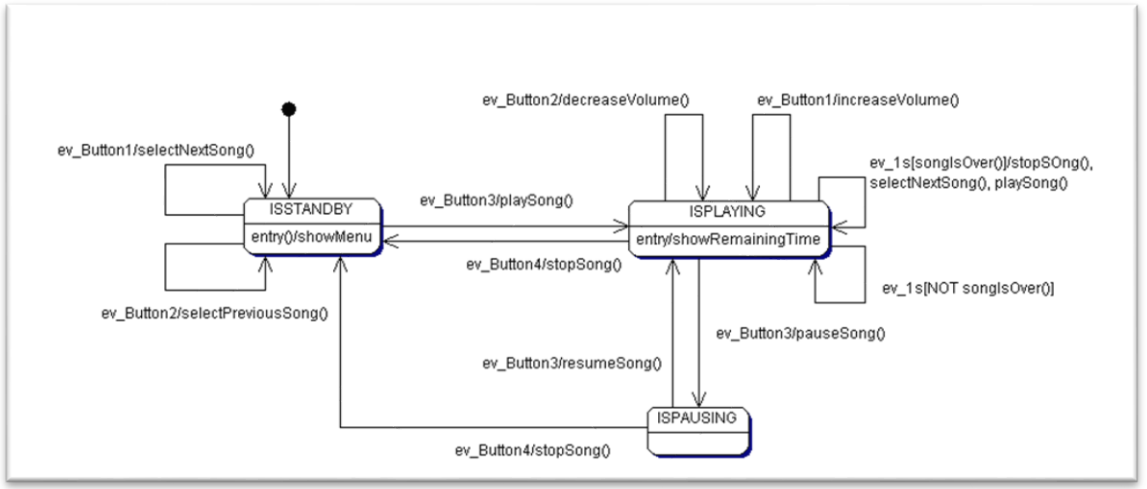
State Machine Design and Implementation

Prof. Dr.-Ing. Peter Fromm



Content

- Motivation State Machines
- State Machine Basics
- Switch-Case Pattern (Revision)
- Look-Up Table
- Special Cases



Car – where we are right now

- We have created the hardware and agreed the elementary software architecture, based on the Autosar RTE concept

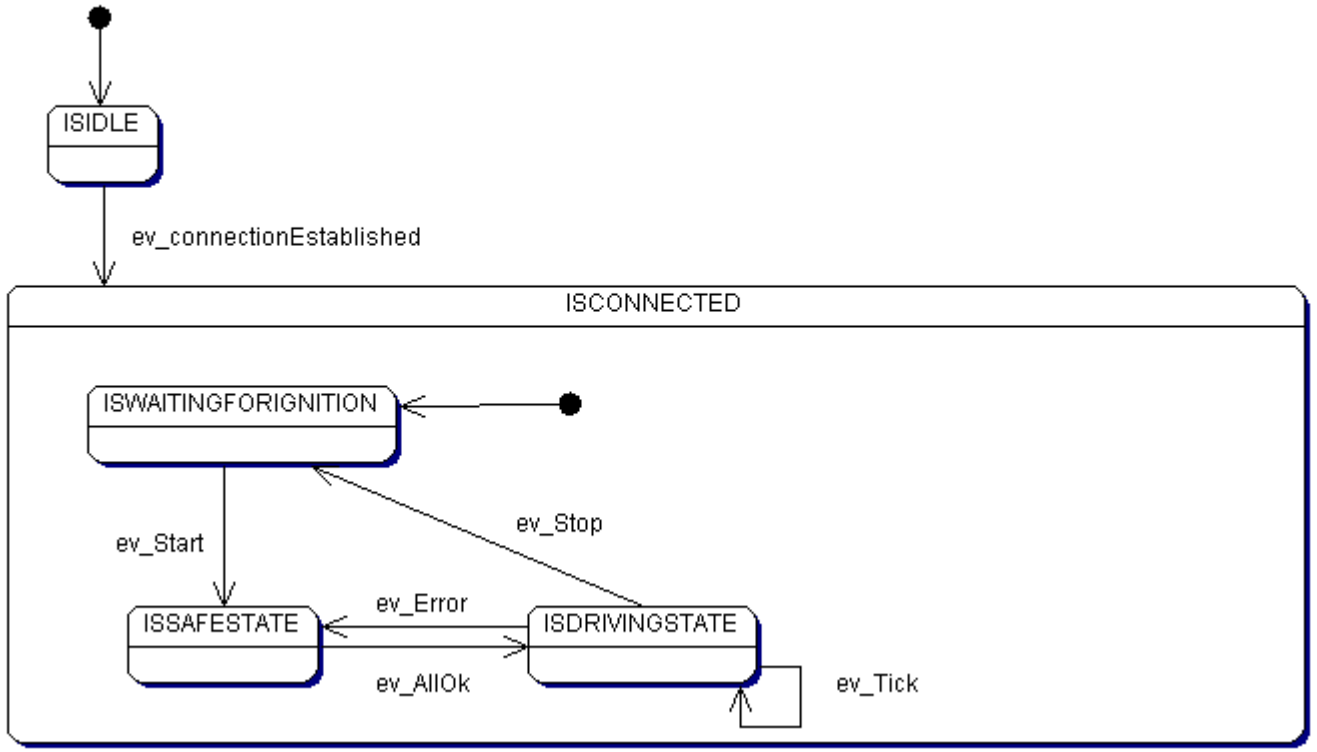
- A complex driver is available for the engine
- Joystick driver for the Labboard is also available
- And the Zigbee communication will be pretty close to a normal UART communication as we already know from the UART_LOG feature

What we have to do

- We have to design and implement a communication protocol
- And we have to think about a system state machine for the car, in order to implement requirements like

Id	Requirement
FR1	The car may only drive, if a valid connection to the control unit is established
FR2	The car should stop and remain in the safe state “stopped” in case more than 2 communication protocols are corrupted
FR3	In case of any other safety requirement reporting a problem (e.g. collision sensor, engine not driving with the intended speed) the car should go into the safe state
FR4	The safe state may only be left if all safety functions report “ok” for 5 times in a row

A first simple state diagram (will be refined later)



Statemachine Definitions

Definition

A finite state machine is an abstract machine that defines a finite set of conditions of existence (called “states”), a set of behaviors or actions performed in each of those states, and a set of events that cause changes in states according to a finite and well-defined rule set.

Bruce Powel Douglass

http://www.embedded.com/1999/9901/9901feat1.htm

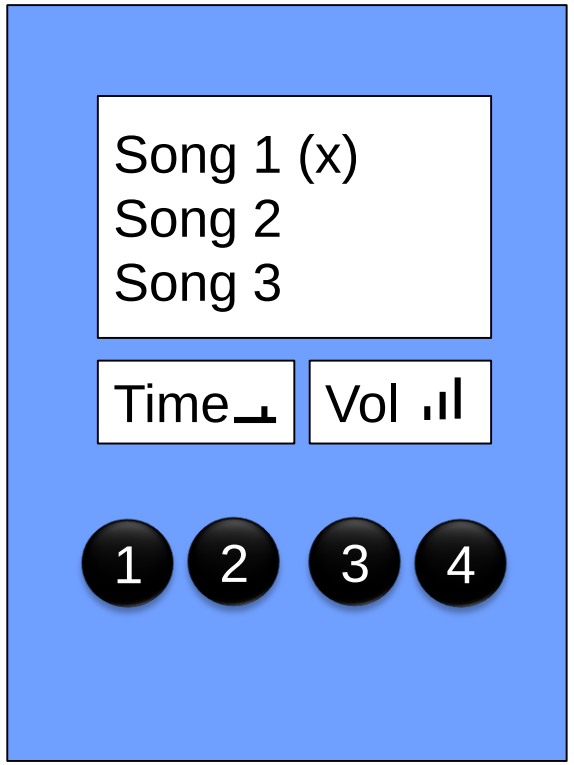
When to use state machines?

- Objects with behavior, that is significant different depending on internal object condition (state, operation mode etc.)
- The state is stored in an internal state variable (object attribute). It is not accessible from outside. State changes **occur only** as (possible) side-effect of event processing.
- Major use case for state machines: reactive systems, e.g.
  - All devices with direct user interactions (MP3 player, mobile phone,...)
  - All devices communicating to other devices via networks (Routers, Printers,....)
  - ...

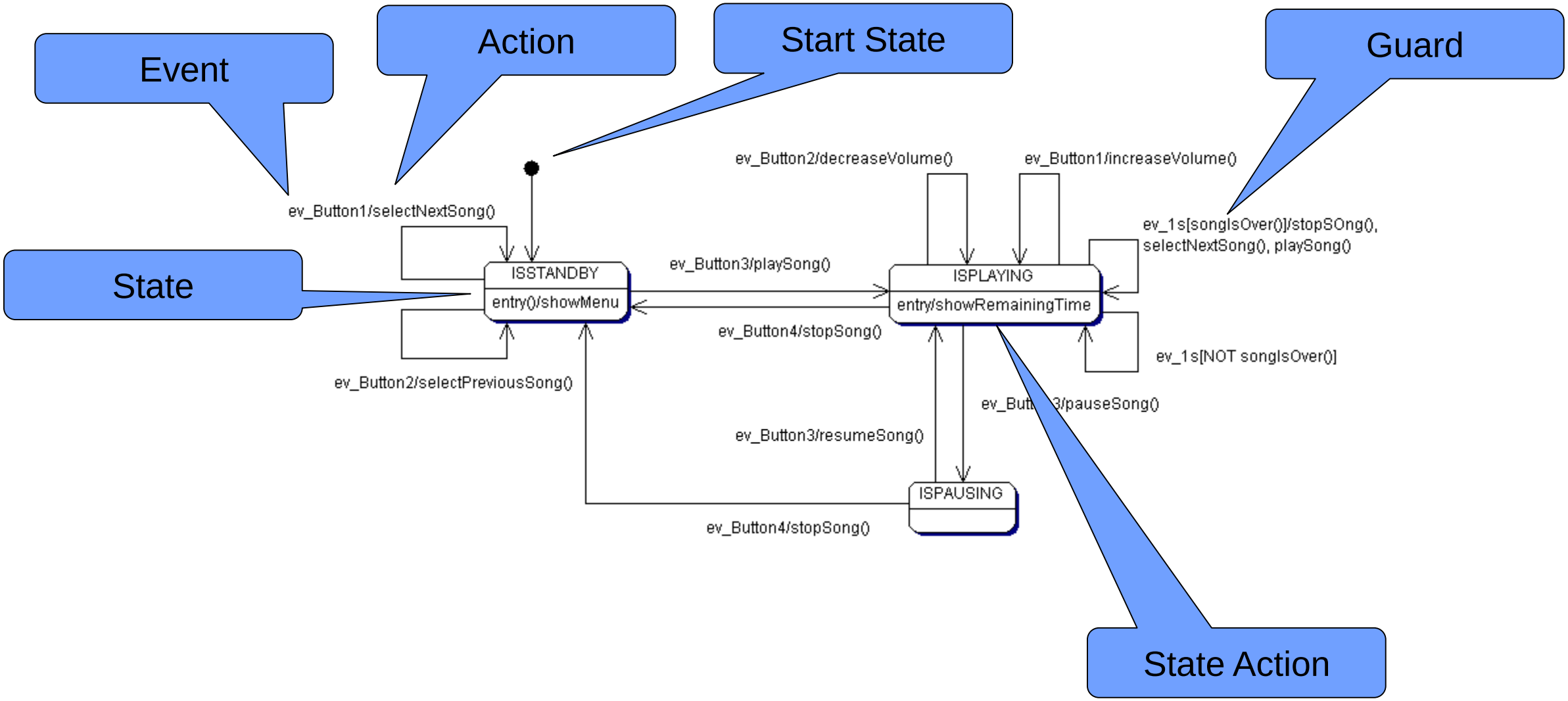
Systems having states operate differently upon the same input, depending on their state.

Our MP3 Player

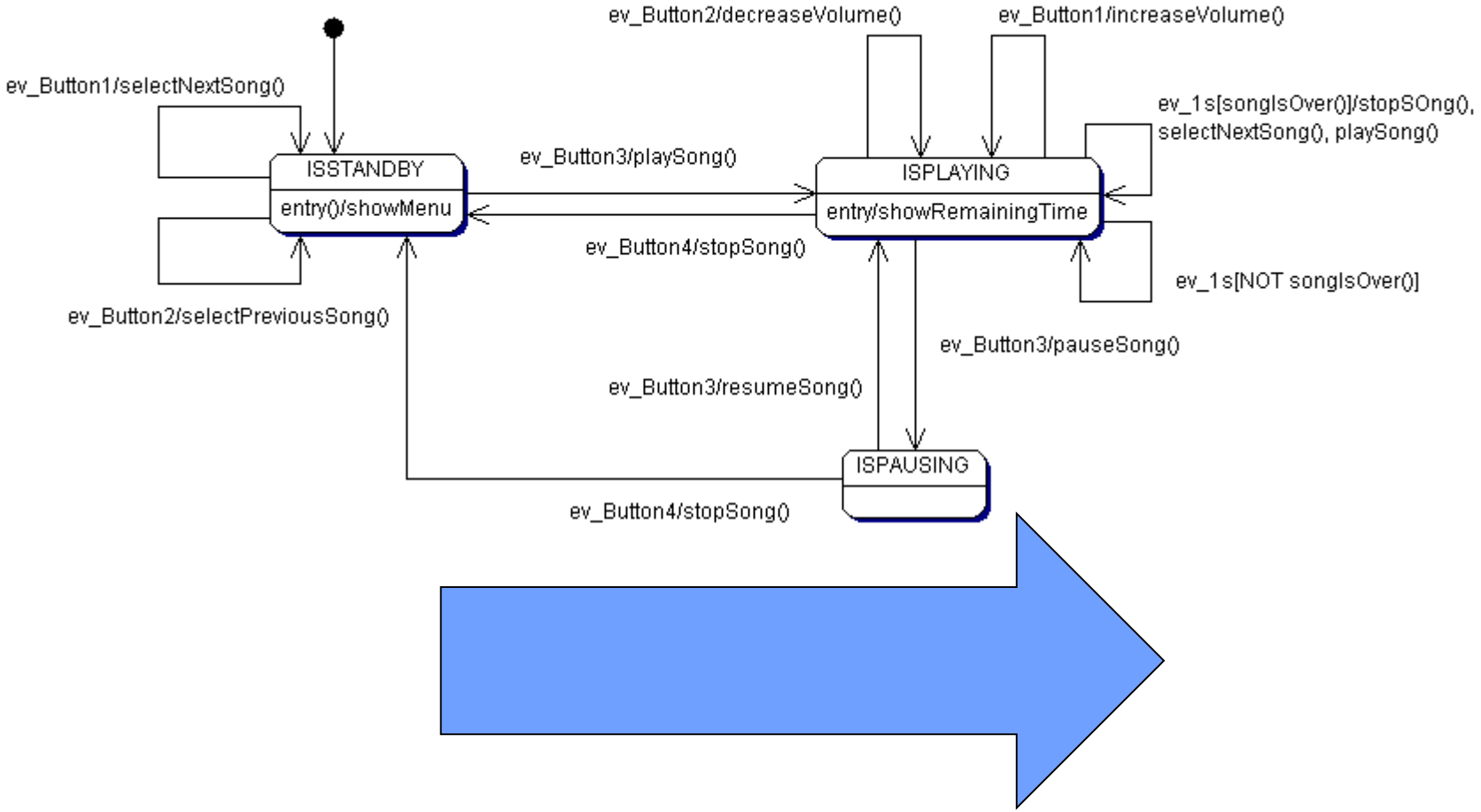
- Our MP 3 Player has 4 buttons
- In the Standby Mode
  - Button_1 and Button_2 are used to select a song
  - Button_3 initiates playing a song
  - Button_4 has no impact
- In the Playing mode
  - Button_1 and Button_2 are used to increase / decrease the volume
  - Button_3 pauses / resumes a song
  - Button_4 terminates playing a song and goes back to the Standby Mode
  - If a song terminates, the next song will start automatically
  - Furthermore, every second the remaining playtime will be shown



A simple State Machine



Design to Code

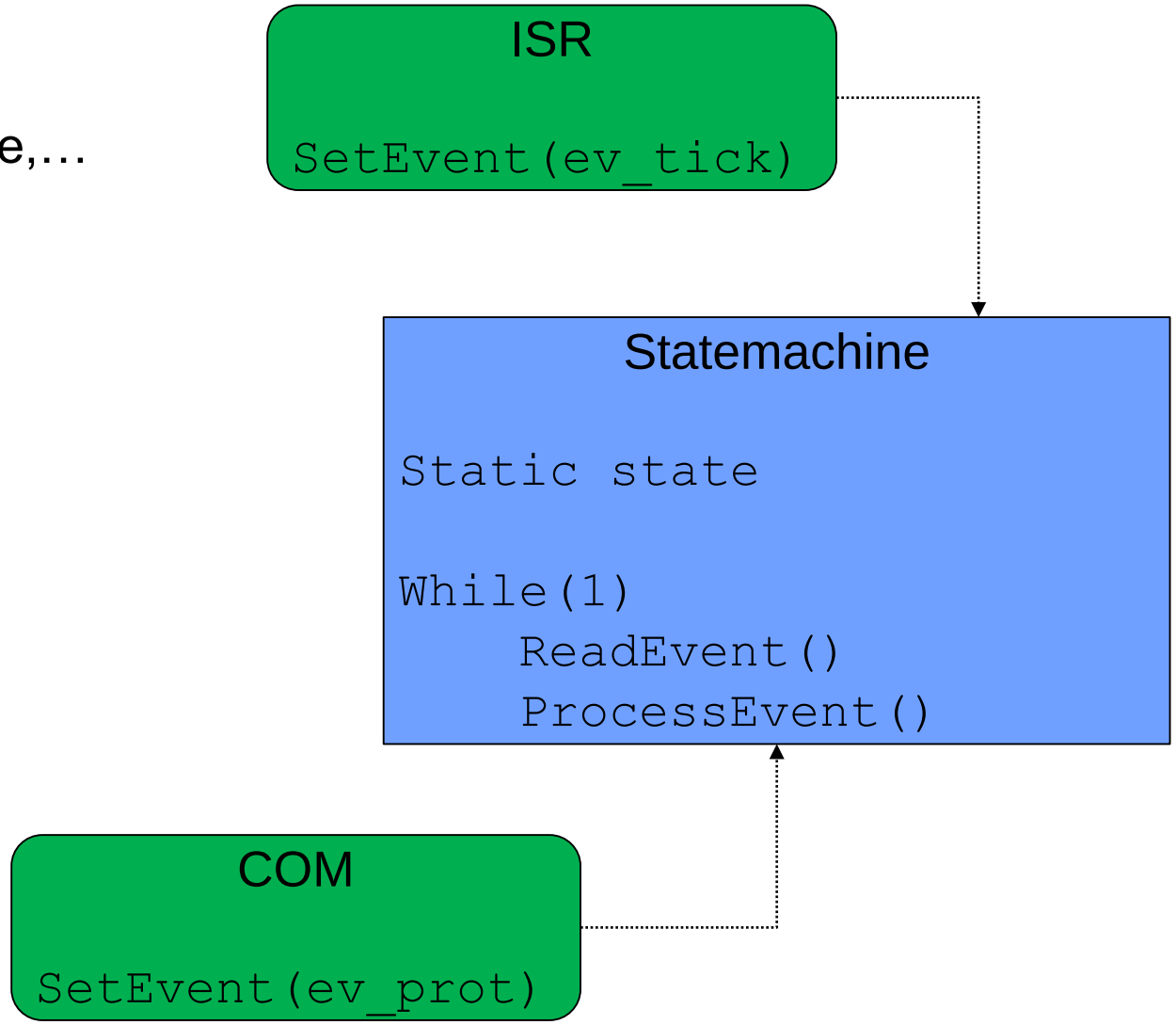


General Implementation Concept

Sender
- Fires Event
- ISR, communication stack, application code,…
- Does not know the behavior / reaction

Receiver (Statemachine)
- Read Event
- Depending on State and Guard Conditions
  - Call action expressions
  - Set new State

- Often implemented as own task
- Task is inactive, as long no event needs to be processed
- Only knows the event, but not the sender



Coding Pattern

There are different solutions how to code a state machine.

- Switch Case (already known)
- Transition Table (different variants – focus of this lecture)
- Extensions towards hierarchical / nested state machines (next lecture)

Basic Coding Pattern - States

```
RC_t MP3_ProcessEvent(EventMaskType nextEvent) {
    switch (MP3__player.m_state)
    {
        case MP3STATE_ISSTOPPED :
            if (nextEvent & ev_Button1)
            {
                MP3__prevSong();
                MP3__showSong();
            }
            else if (nextEvent & ev_Button2)
            {
                MP3__nextSong();
                MP3__showSong();
            }
            else if (nextEvent & ev_Button3)
            {
                MP3__playSong();
                MP3__showSong();
                MP3__player.m_state = MP3STATE_ISPLAYING;
            }
            break;

        case MP3STATE_ISPLAYING :
                //…
```

States

Usually the states act as cases, as this makes the structure easier to understand, as compared to using the events as case.

Basic Coding Pattern - Events

```
RC_t MP3_ProcessEvent(EventMaskType nextEvent) {
    switch (MP3__player.m_state)
    {
        case MP3STATE_ISSTOPPED :
            if (nextEvent & ev_Button1)
            {
                MP3__prevSong();
                MP3__showSong();
            }
            else if (nextEvent & ev_Button2)
            {
                MP3__nextSong();
                MP3__showSong();
            }
            else if (nextEvent & ev_Button3)
            {
                MP3__playSong();
                MP3__showSong();
                MP3__player.m_state = MP3STATE_ISPLAYING;
            }
            break;

        case MP3STATE_ISPLAYING :
                //…
```

Events

The events usually are checked using an if – else if structure, ensuring that only one event is being processed. Check the notes on bitmask events later in this lecture.

Basic Coding Pattern - Actions

```
RC_t MP3_ProcessEvent(EventMaskType nextEvent) {
    switch (MP3__player.m_state)
    {
        case MP3STATE_ISSTOPPED :
            if (nextEvent & ev_Button1)
            {
                MP3__prevSong();
                MP3__showSong();
            }
            else if (nextEvent & ev_Button2)
            {
                MP3__nextSong();
                MP3__showSong();
            }
            else if (nextEvent & ev_Button3)
            {
                MP3__playSong();
                MP3__showSong();
                MP3__player.m_state = MP3STATE_ISPLAYING;
            }
            break;

        case MP3STATE_ISPLAYING :
                //…
```

Actions

Depending on the state and event (transitions), the corresponding action functions are being called. It is possible to call more than one action in the same transition.
Actions from the transition itself but also entry/exit actions from the state must be considered.

Basic Coding Pattern – State Change

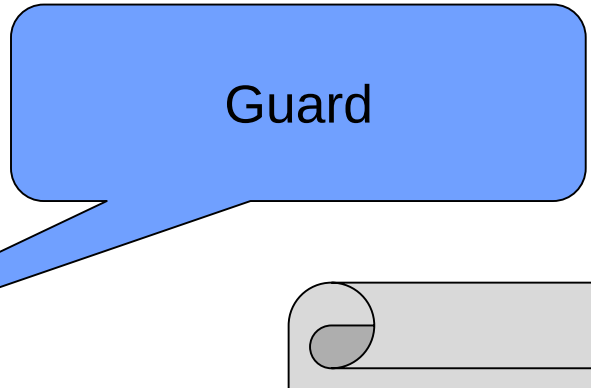
```
RC_t MP3_ProcessEvent(EventMaskType nextEvent) {
    switch (MP3__player.m_state)
    {
        case MP3STATE_ISSTOPPED :
            if (nextEvent & ev_Button1)
            {
                MP3__prevSong();
                MP3__showSong();
            }
            else if (nextEvent & ev_Button2)
            {
                MP3__nextSong();
                MP3__showSong();
            }
            else if (nextEvent & ev_Button3)
            {
                MP3__playSong();
                MP3__showSong();
                MP3__player.m_state = MP3STATE_ISPLAYING;
            }
            break;

        case MP3STATE_ISPLAYING :
                //…
```

State Change

Depending on the state and event (transitions), the corresponding action functions are being called. It is possible to call more than one action in the same transition.

Basic Coding Pattern – Guards

```
case MP3STATE_ISPLAYING :
            if (nextEvent & ev_Button1)
            {
                MP3__increaseVolume();
            }
             else if (…)
            {
                 …
            }
            else if (nextEvent & ev_1s && !MP3__songEnded())
            {
                MP3__showRemainingPlaytime();
            }
            else if (nextEvent & ev_1s && MP3__songEnded())
            {
                MP3__nextSong();
                MP3__playSong();
                MP3__showSong();
            }
            break;
```



If a guard is defined, the transition will only be fired if the guard is true.

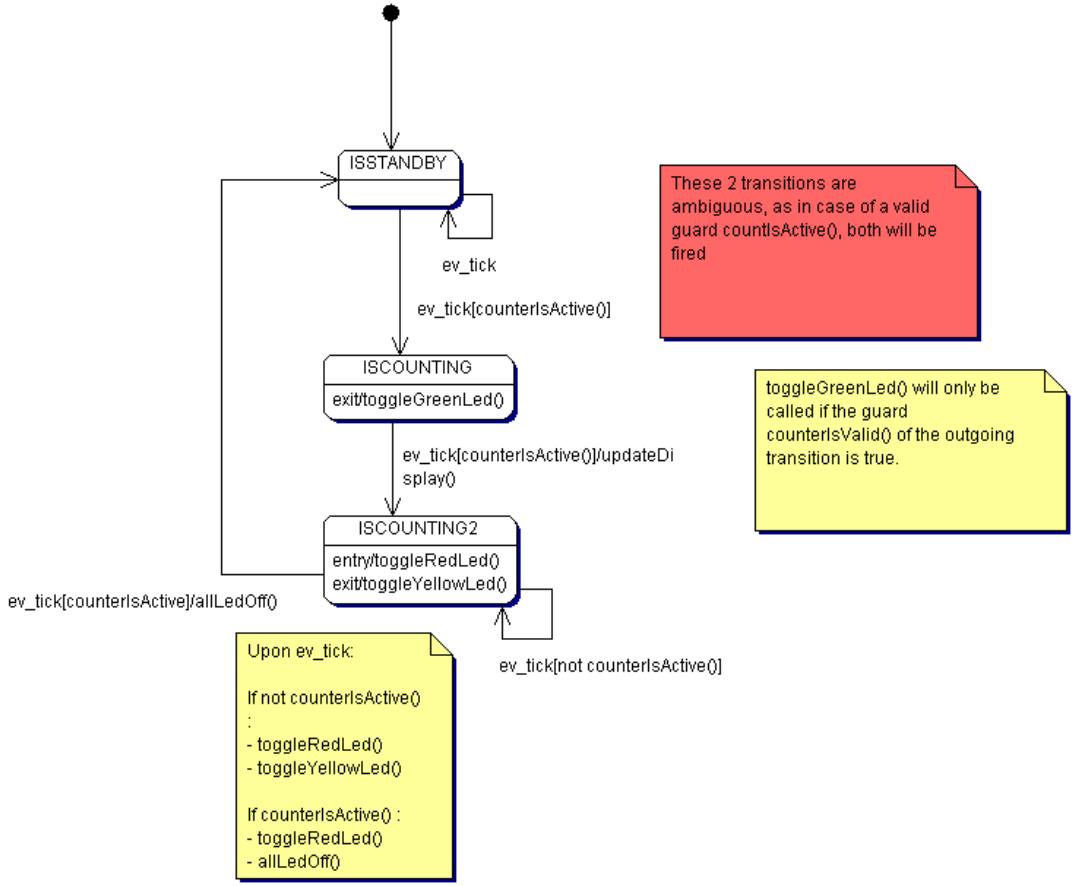
Action Sequence and Guard Considerations

General sequence

- Current state exit
- Transition action
- New state entry

Guards

- Guards may not be ambiguous, i.e. it may not be possible to fire 2 transitions in case of a single event.
- If a transition is not fired due to a failed guard, it is a good design to also not fire the state exit function.
- If the state exit should be fired, add a self transition with a negative guard.



Do actions are not used, as their execution trigger is not really defined.

Evaluation Switch Case Pattern

Pro’s

- Simple to use
- Very flexible
- Supports any number of guards, transitions and actions
- Straight forward debugging
- Pretty fast

Cons

- State machine logic not easy visible
- Code may become complex and hacky
- Actions do not need to be own functions

Good for small and flat statemachines.

Lookup Table

Key Idea

- Every graphical state machine implementation can be described in a tabular format.
- Every record in this table describes a single transition.

Elements

- Data structure containing representing transitions and state actions
- Table containing the Action, Guard and target state pointers
- A loop checking the states and calling the corresponding functions

FromState	Event	ToState	Guard	Action
ISSTANDBY	Ev_Button1	ISSTANDBY	0	nextSong()
ISSTANDBY	Ev_Button2	ISSTANDBY	0	prevSong()
ISSTANDBY	Ev_Button3	ISPLAYING	0	playSong()

Remember the function pointer story?

```
/**
 * function pointer used for actions
 */
typedef void (*STATE_ActionPtr_t)();

/**
 * function pointer used for guards
 */
typedef boolean_t (*STATE_GuardPtr_t)();
```

Design Concept 1 – Full Transition Table

- The transition table stores the complete information, i.e.
  - FromState, Event, To State
  - Guard and Action Function Pointer
- A second table stores the states entry and exit actions

```
/** State Transition Table Entry */
typedef struct
{
          STATE_State_t fromState;
          STATE_State_t toState;
          STATE_Event_t event;
          STATE_GuardPtr_t guard;
          STATE_ActionPtr_t actionTransition;
} STATE_stateTransition_t;


/** State Action */
typedef struct
{
          STATE_State_t state;
          STATE_ActionPtr_t actionEntry;
          STATE_ActionPtr_t actionExit;
} STATE_stateAction_t;
```

Design Concept 1 – Full Transition Table
Example Alive Watchdog controlling a FailSafe Mode

```
const STATE_stateTransition_t STATE_Alive_Transitions[] = {
/* FromState     Event            ToState          Guard                       Action            */
{ state_ok,      ev_tick,         state_ok,        STATE__Alive_Timeout_OK,   STATE__tick      },
{ state_ok,      ev_tick,         state_error,     STATE__Alive_Timeout_NOK, 0,                },
{ state_ok,      ev_trigger,      state_ok,        STATE__Alive_Timeout_OK,   STATE__resetTimer},
{ state_ok,      ev_trigger,      state_error,     STATE__Alive_Timeout_NOK, 0                 },
{ state_ok,      ev_restart,      state_ok,        0,                         STATE__resetTimer},

{ state_error,   ev_tick,         state_error,     0,                         STATE_tick       },
{ state_error,   ev_restart,      state_ok,        0,                         STATE_resetTimer },

{ 0, 0, 0, 0, 0} /* End of table record - must be unique */
};

const STATE_stateAction_t STATE_Alive_StateActions[] = {
      /* State            Entry                      Exit */
    {    state_ok,        STATE__PowerOn,            0     },
    {    state_error,     STATE__PowerOff,           0     },

    { 0, 0, 0} /* End of table record - must be unique */

};
```

Design Concept 1 – Full Transition Table
Pseudo Code Statemachine Implementation

```
void processEvent(event_t ev)

        state_t currentState = getState()

        while (not endOfTransitionList)
                transition = transition_table[i++];
                if (transition.fromState == currentState && transition.event = ev)

                        state_t toState = transition.toState
                        guard_t guard = transition.guard
                        action_t action = transition.action

                        if (guard == NULL || guard()==TRUE)
                                while (not endOfStateActionList)
                                        stateActions = action_table[j++]
                                        if (stateActions.state == toState)
                                                entry = stateActions.entry;
                                        if (stateActions.state == currentState)
                                                exit = stateActions.exit;

                        if (exit != NULL) exit();
                        if (action != NULL) action();
                        if (entry != NULL) entry;

                        setState(toState)
```

Design Concept 1 – Full Transition Table Evaluation

Pro’s

- Very close to the graphical design
- Good readability
- Very flexible, supports any number of guards and actions
- Good memory footprint for state machines having low number of transitions, as empty transitions are not stored

Con’s

- Bad performance, as the algorithm has to iterate through the complete list for every event to find possible transitions
- Bad memory footprint for FSM’s having many transitions as fromState and Event are explicitly stored
- Entry and Exit actions are limited to a single action (unless a dynamic list is used)

Design Concept 2 – Explicit Transition Table

- The transition table stores the a subset of the information, i.e.
  - To State
  - Guard and
  - Action Function Pointer
- A second table stores the states entry and exit actions
- The index in the table can be calculated using the fromState and event id
- Also empty transitions need to be added explicitly

```
/** State Transition Table Entry */
typedef struct
{
          STATE_State_t toState;
          STATE_GuardPtr_t guard;
          STATE_ActionPtr_t actionTransition;
} STATE_stateTransition_t;

/** State Action */
typedef struct
{
          STATE_ActionPtr_t actionEntry;
          STATE_ActionPtr_t actionExit;
} STATE_stateAction_t;
```

Design Concept 2 – Explicit Transition Table
Example Alive Watchdog controlling a FailSafe Mode

```
const STATE_stateTransition_t STATE_Alive_Transitions[] = {
/* FromState      Event             ToState          Guard                        Action           */
  /* state_ok,    ev_tick, */       /*   state_ok,    STATE__Alive_Timeout_OK,    STATE__tick */
{ /* state_ok,    ev_tick, */       state_error,     STATE__Alive_Timeout_NOK,   0                },
{ /* state_ok,    ev_trigger, */    state_ok,        STATE__Alive_Timeout_OK,    STATE__resetTimer},
{ /* state_ok,    ev_restart, */    state_ok,        0,                          STATE__resetTimer},

{ /* state_error,ev_tick, */       state_error,     0,                          STATE_tick       },
{ /* state_error,ev_trigger, */    state_error,     0,                          0},
{ /* state_error,ev_restart, */    state_ok,        0,                          STATE_resetTimer },

};

const STATE_stateAction_t STATE_Alive_StateActions[] = {
      /* State             Entry                        Exit */
    { /* state_ok, */      STATE__PowerOn,              0     },
    { /* state_error, */   STATE__PowerOff,             0     },
};
```

Note: As only exactly one transition is possible per state/event combination, the first transition has to be implemented in a different way – might be hacky…

Design Concept 2 – Explicit Transition Table
Pseudo Code Statemachine Implementation

```
void processEvent(event_t ev)

        state_t currentState = getState()

        uint indexTransition = currentState * numberEvents + ev

        transition = transition_table[indexTransition];

        state_t toState = transition.toState
        guard_t guard = transition.guard
        action_t action = transition.action

        exit = action_table[currentState]
        entry = action_table[toState]

        if (guard == NULL || guard()==TRUE)
                if (exit != NULL) exit();
                if (action != NULL) action();
                if (entry != NULL) entry;

                setState(toState)
```

Calculate index

Execute actions

Set new state

Design Concept 2 – Explicit Transition Table Evaluation

Pro’s

- Very good performance, as index can be calculated
- Good readability
- Good memory footprint for state machines having many transitions, as fromState and Event are not stored

Con’s

- Not very flexible, only supports one guards and action per event/state
- Bad memory footprint for FSM’s having few transitions as empty transitions are explicitly stored

Design Concept 3 – Two Layer Transition Table

- We use two different layers for the transition table
- The inner table stores the list of transitions **per state**
  - Event, ToState, Guard and Action Function Pointer
- The outer transition table stores the pointers to the inner tables
- Similar concept for the state action table

```
/** Inner State Transition Table Entry */
typedef struct
{
          STATE_Event_t event;
          STATE_State_t toState;
          STATE_GuardPtr_t guard;
          STATE_ActionPtr_t actionTransition;
} STATE_innerTransition_t;

/** Outer State Transition Table Entry */
typedef struct
{
 STATE_state_t   fromState;
 STATE_stateInnerTransitionTable_t const *  const
     pInnerTransitionTable;
     uint16_t const size;
} STATE_stateOuterTransition_t;
```

Transitions leaving a specific state are stored in one table, i.e. we need one inner table per state.

The outer table collects all internal tables in a common object.

Design Concept 3 – Two Layer Transition Table
Example Alive Watchdog controlling a FailSafe Mode

```
const STATE_innerTransition_t STATE_state_ok[] = {
        /*  Event         ToState          Guard                       Action           */
        {   ev_tick,      state_ok,        STATE__Alive_Timeout_OK,  STATE__tick      },
        {   ev_tick,      state_error,     STATE__Alive_Timeout_NOK, 0                },
        {   ev_trigger,   state_ok,        STATE__Alive_Timeout_OK,  STATE__resetTimer},
        {   ev_reset,     state_ok,        0,                        STATE__resetTimer},
};
const STATE_innerTransition_t STATE_state_error[] = {
        /*  Event         ToState          Guard                       Action           */
        { ev_tick,        state_error,     0,                         STATE_tick       },
        { ev_restart,     state_ok,        0,                         STATE_resetTimer },
};

const STATE_stateOuterTransitionTable_t STATE_Alive_Transitions = {
/*    fromState          Pointer to table                      Size of table [Elements]    */
    { state_ok,          & STATE_state_ok,
sizeof(STATE_state_ok)/sizeof(STATE_stateInnerTransition_t)    },

    { state_warn,        0,                                    0    },

    { state_error,       & STATE_state_error,
sizeof(STATE_state_error)/sizeof(STATE_stateInnerTransition_t) },
};
```

Design Concept 3 – Two Layer Transition Table Implementation

- Similar like the full transition table, but the lists which need to be iterated are shorter, as only the list belonging to the `fromState` needs to be checked.

- Note the use of sizeof to automatically calculate the size of the inner tables and to store it in the outer table.

Depending on the requirements and constraints, different implementations are possible.

- Only one state machine or active objects
- Events as bit pattern or enums, queued or unqueued
- Individual actions or “sibling” transitions
- Internal handling of the state variable or provision as external object

Sample API

```
/**
 * This function will process the event with the defined state machine
 * The state machine is implemented as a singleton object, only the configuration may differ
 * \param STATE_stateTransitionTable_t const * const transitionTable    : IN - Pointer to the transition table
 * \param uint16_t transitionTableSize                                  : IN - Size of the transition table
 * \param STATE_stateActionTable_t const * const actionTable            : IN - Pointer to the action table
 * \param STATE_event_t ev                                              : IN - Event to be processed
 * \param  STATE_state_t* state                                         : IN/OUT - New State
 * \return RC_SUCCESS or error
 */
RC_t STATE_processEvent(STATE_stateOuterTransitionTable_t const * const  transitionTable,
                        uint16_t transitionTableSize,
                        STATE_stateOuterAction_t const* const actionTable,
                        STATE_event_t ev,
                        STATE_state_t* state)
{
```

Outer Table handling

```
STATE_GuardPtr_t guard = 0;
STATE_ActionPtr_t acTransition = 0;
STATE_ActionPtr_t acEntry = 0;
STATE_ActionPtr_t acExit = 0;


//Read the state from the watchdog object;
STATE_state_t currentState = *state;
STATE_state_t toState = currentState; //Just in case

//Calculate the table index of the outer table
sint16_t index = STATE__getIndexOfState(transitionTable, transitionTableSize, currentState);

//This should never happen
if (STATE_INVALIDINDEX == index)
{
    return RC_ERROR_RANGE;
}

STATE_stateInnerTransitionTable_t const* const pInnerTable = (*transitionTable)[index].pInnerTransitionTable;

uint16_t innerTableSize = 0;
if (0 != pInnerTable)
{
    innerTableSize = (*transitionTable)[index].size;
}

if (0 == pInnerTable || 0 == innerTableSize)
{
    //No transitions in this state
    return RC_ERROR_INVALID_STATE;
}
```

Inner Table Handling

```
//Iterate through the inner table and find correct transition
for (uint16_t i = 0; i < innerTableSize; i++)
{
    if ((*pInnerTable)[i].event == ev)
    {
        //Transition found, let's check the guard
        guard = (*pInnerTable)[i].guard;
        if (0 == guard || TRUE == guard())
        {
            //Transition is valid
            acTransition = (*pInnerTable)[i].actionTransition;
            toState = (*pInnerTable)[i].toState;

            //Now lets find the entry and exit actions
            uint16_t actionTableSize = actionTable->size;
            for (uint16_t j = 0; j < actionTableSize; j++)
            {
                if (currentState == (*actionTable->pInnerStateTable)[j].state)
                {
                    acExit = (*actionTable->pInnerStateTable)[j].actionExit;
                }
                if (toState == (*actionTable->pInnerStateTable)[j].state)
                {
                    acEntry = (*actionTable->pInnerStateTable)[j].actionEntry;
                }
            }

            //Lets call the actions
            if (0 != acExit) acExit();
            if (0 != acTransition) acTransition();
            if (0 != acEntry) acEntry();



            //If we want to fire only one transition, the break must be actiavted
            //Note: More than one transition / action only make sense, if the taregt states are identical
            //This must be ensured by the user
            //break;
        }
    }
}

//Copy local toState to external State Variable
*state = toState;


return RC_SUCCESS;
```

Design Concept 3 – Two Layer Transition Table Evaluation

Pro’s

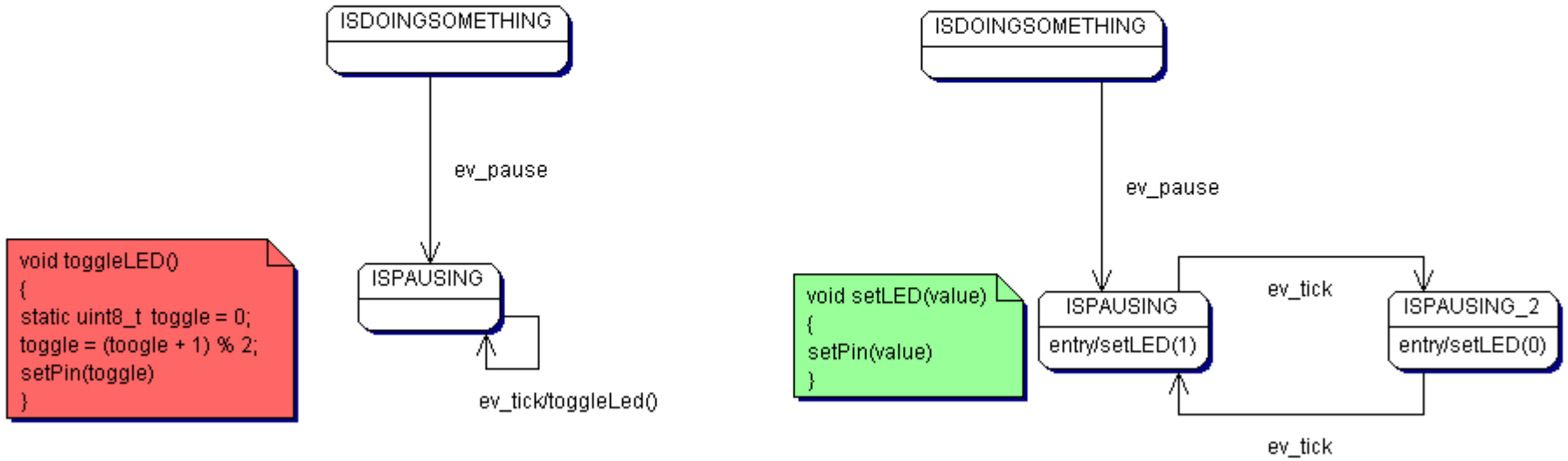
- Medium performance, as only one index can be calculated
- Good readability
- Very flexible, supports any number of guards/actions
- Good memory footprint for any state machine

Con’s

- Complex implementation

Flag versus additional State

- Very often, developers tend to add extra flags into the functions to memorize a sub-state.
- A simple example is shown below, which lets a LED blink in case of a pause state.
- Although the left design looks easier, it actually is more complex, as we have another static variable in the game, which might cause hard to debug races.



Enum versus Bitmask Events

- All presented algorithms work fine and without modification if the event comes as enum type

- Operating systems however often use bitmasks for representing events
  - More than one event may be fired at the same time
  - Rather than the bitmask value, we need the position of the bit for further processing

- Solution: we have to clock the bits out of the mask

```
myEvent = 0;
while(0 != evmask)
        for (i = 0; i < bitsize(evmask); i++)
                if (evmask & 1 << i)           //Bit was set
                        myEvent = i;           //Store the position
                        evmask &= ~(1<< i);    //Clear the bit
                        break
```

WrapUp - Which one to choose?

	Switch Case	Full Transition	Explicit Transition	Two Layer
Performance	++	--	++	+
Flexibility	++	++	- (3)	++
Maintainability	--	++	+/-	++
Memory Footprint	++	+/-	+/-	+
Implementation	+ (1)	- (2)	+/-	- (2)

(1) Good for very simple state machines
(2) State machine code is complex, but can be used for different implementations. Maintaining the table is simple.
(3) For many statemachines, a single transition per state/event is sufficient